JAVA GENERICS - COMPREHENSIVE EXAM GUIDE

Complete Study Material for Beginners

---

1. INTRODUCTION TO GENERICS

What are Generics?

Generics provide a way to write classes, interfaces, and methods that can work with different data types while maintaining type safety at compile time. They allow you to specify the type of objects that a collection or class can hold, preventing ClassCastException runtime errors. Generics were introduced in Java 5 (J2SE 5.0) to eliminate unsafe casting and to provide compile-time type checking.

Why Do We Need Generics?

Before Generics (Without Type Safety):

```
java

List list = new ArrayList();
list.add("Hello");
list.add(123);  // Compiler allows this!

String str = (String) list.get(1);  // Runtime Error!
// ClassCastException because 123 is not a String
```

The problem is that the compiler doesn't prevent type errors. You can add any type to the list, and when you retrieve an element, you must cast it. If the cast fails, you get a runtime exception.

With Generics (Type Safe):

```
java

List<String> list = new ArrayList<String>();
list.add("Hello");
list.add(123);  // Compilation Error! Caught early.

String str = list.get(0);  // No casting needed!
// Type is automatically String
```

Now the compiler knows list only contains Strings. Errors are caught at compile time, not runtime.

Key Benefits of Generics:

1. **Type Safety**: Errors are caught at compile time, not runtime.
2. **Elimination of Casts**: No need for explicit type casting when retrieving elements.

3. **Code Reusability**: Write generic code that works with any data type.
4. **Compile-time Checking**: Type mismatches are caught before code runs.
5. **Performance**: No need for runtime type checking or casting overhead.

---

2. SIMPLE GENERICS EXAMPLES

Understanding Generic Syntax

A generic type is declared with angle brackets `<>` containing one or more type parameters. Type parameters are placeholders for actual types that will be specified when using the class.

Example 1: Simple Generic Box Class

```java
public class Box<T> {
    private T item;

    public void set(T item) {
        this.item = item;
    }

    public T get() {
        return item;
    }
}
```

In this example:

- `<T>` is a type parameter. T stands for "Type" (can be any letter or name).
- The class can store and retrieve objects of any type.
- T is replaced with the actual type when the class is instantiated.

Using the Generic Box Class

```java
```

```
// Creating Box for String
Box<String> stringBox = new Box<String>();
stringBox.set("Hello Generics");
String str = stringBox.get();  // No casting!

// Creating Box for Integer
Box<Integer> intBox = new Box<Integer>();
intBox.set(42);
Integer num = intBox.get();

// This will cause COMPILE TIME ERROR:
stringBox.set(123);  // Cannot add Integer to String Box
```

Explanation:

- When you write `Box<String>`, you're telling the compiler that this Box will hold only String objects.
- The compiler ensures type safety and prevents mixing different types.
- No casting is required when retrieving elements because the type is known.

Example 2: Generic Pair Class with Multiple Type Parameters

java

```
public class Pair<K, V> {
    private K key;
    private V value;

    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() { return key; }
    public V getValue() { return value; }
}
```

Usage:

java

```
Pair<String, Integer> pair = new Pair<>("Age", 25);
System.out.println(pair.getKey());    // Output: Age
System.out.println(pair.getValue()); // Output: 25
```

Note: You can omit the type parameters in the constructor call with the diamond operator `<>` (Java 7+).

3. GENERIC TYPES

What are Generic Types?

A generic type is a generic class or interface that is parameterized over types. When you create an instance of a generic type by specifying actual type arguments, you get a parameterized type.

Type Parameters and Type Arguments

- **Type Parameter**: The placeholder declared in angle brackets, like `<T>` in `public class Box<T>`. It is abstract and defined at class/interface definition time.
- **Type Argument**: The actual type provided when using the generic type, like `String` in `Box<String>`. It is concrete and provided when instantiating.

Type Parameter	Type Argument
`class Box<T>` (abstract)	`new Box<String>` (concrete)

Generic Interfaces

Interfaces can also be generic. A class implementing a generic interface must specify type parameters.

```java
public interface Container<T> {
    void put(T item);
    T get();
}

public class MyContainer<T> implements Container<T> {
    private T item;
    public void put(T item) { this.item = item; }
    public T get() { return item; }
}
```

When implementing a generic interface, you can either:

1. **Keep it generic**: `public class MyContainer<T> implements Container<T>`
2. **Specify a concrete type**: `public class StringContainer implements Container<String>`

4. GENERIC METHODS

What are Generic Methods?

A generic method is a method that introduces its own type parameters. These methods can be called with different types, and the type parameter is determined by the argument or return type at the call site. Generic methods can exist in both generic and non-generic classes.

Syntax of Generic Methods

```java
public <T> T genericMethod(T input) {
    // Method body
    return input;
}
```

Important: The type parameter `<T>` appears **BEFORE** the return type. This tells the compiler that T is a type parameter of the method.

Example 1: Simple Generic Method

```java
public class Utils {
    public static <T> void printArray(T[] array) {
        for (T element : array) {
            System.out.println(element);
        }
    }
}

// Usage:
Utils.printArray(new Integer[]{1, 2, 3});
Utils.printArray(new String[]{"a", "b", "c"});
```

Explanation: The method `printArray` can accept an array of any type. The type parameter T is determined by the type of array passed.

Example 2: Generic Method with Multiple Type Parameters

```java
```

```
public static <K, V> void printMap(K key, V value) {
    System.out.println("Key: " + key);
    System.out.println("Value: " + value);
}

// Usage:
printMap("Name", "John");
printMap(1, 2.5);
```

Key Differences: Generic Classes vs Generic Methods

Generic Class	Generic Method
Type parameter declared at class level	Type parameter declared at method level
Applies to all methods in class	Applies only to that specific method
`public class Box<T>`	`public <T> T method()`

5. BOUNDED TYPE PARAMETERS

What are Bounded Type Parameters?

By default, generic type parameters can be any type. Bounded type parameters restrict the types that can be used. You can specify an upper bound using the keyword 'extends' to limit which types can be passed as type arguments.

Why Use Bounded Type Parameters?

1. **Type Safety**: Ensure only compatible types are used.
2. **Call Restricted Methods**: Access methods of the bound class.
3. **Compile-time Checking**: Errors caught early.

Upper Bounded Wildcard with extends

```
java
```

```java
public class NumberBox<T extends Number> {
    private T value;

    public void setValue(T value) {
        this.value = value;
    }

    public double toDouble() {
        return value.doubleValue();  // Can call Number's methods
    }
}
```

Usage and Explanation:

```java
NumberBox<Integer> intBox = new NumberBox<>();
intBox.setValue(42);
System.out.println(intBox.toDouble()); // 42.0

NumberBox<Double> doubleBox = new NumberBox<>();
doubleBox.setValue(3.14);

// This will give COMPILE TIME ERROR:
NumberBox<String> stringBox = new NumberBox<>(); // Error!
// String is not a subclass of Number
```

Explanation:

- `<T extends Number>` means T can only be a subclass of Number (Integer, Double, Float, etc.).
- This allows us to call Number's methods like `doubleValue()`.
- Any attempt to use a type that doesn't extend Number will fail at compile time.

Multiple Bounds

A type parameter can have multiple bounds using the '&' operator. The first bound is a class, and the rest are interfaces.

```java
public class MyClass<T extends Number & Comparable<T>> {
    // T must be a subclass of Number AND implement Comparable
}
```

Explanation: T must satisfy both Number and Comparable. Class bounds come first, then interfaces separated by `&`.

6. WILDCARDS

What are Wildcards?

A wildcard is a special type denoted by '?' that represents an unknown type. Wildcards are used in generic code to provide flexibility when you don't know the exact type at compile time. They are typically used in method parameters, return types, and variable declarations.

Types of Wildcards

1. Unbounded Wildcard: `<?>`

Represents any type. Used when you want to work with any type of parameterized class.

```java
public static void printList(List<?> list) {
    for (Object obj : list) {
        System.out.println(obj);
    }
}

// Can call with any type:
printList(new ArrayList<String>());
printList(new ArrayList<Integer>());
```

Limitation: With unbounded wildcard, you can only read (get) elements, not write (add).

```java
List<?> list = new ArrayList<String>();
list.add("Hello");  // COMPILE ERROR - can't add to unknown type
String str = (String) list.get(0); // OK - reading is allowed
```

2. Upper Bounded Wildcard: `<? extends Type>`

Accepts any type that is a subclass of the specified type. Used when you need to read from a collection and know the minimum type.

```java
```

```
public static void sumNumbers(List<? extends Number> list) {
    double sum = 0;
    for (Number n : list) {
        sum += n.doubleValue();
    }
    System.out.println("Sum: " + sum);
}

// Usage:
sumNumbers(Arrays.asList(1, 2, 3));          // Integer
sumNumbers(Arrays.asList(1.5, 2.5, 3.5));    // Double
```

Limitation: Cannot add new elements because the exact type is unknown.

```
java

List<? extends Number> list = new ArrayList<Integer>();
list.add(5);  // COMPILE ERROR - don't know if it's Integer, Double, etc.
Number n = list.get(0); // OK - reading is allowed
```

3. Lower Bounded Wildcard: `<? super Type>`

Accepts the type and any superclass of it. Used when you need to write to a collection.

```
java

public static void addNumbers(List<? super Integer> list) {
    list.add(1);
    list.add(2);
    list.add(3);
}

// Usage:
List<Number> numberList = new ArrayList<>();
addNumbers(numberList); // Integer is subclass of Number
```

Limitation: Cannot reliably read elements because you don't know if they are Integers or other Numbers.

```
java

List<? super Integer> list = new ArrayList<Number>();
list.add(5);        // OK - Integer can be added to Number list
Number n = list.get(0);  // COMPILE ERROR - doesn't know the type
Object obj = list.get(0); // OK - everything is Object
```

PECS Rule (Producer Extends, Consumer Super)

- Use `<? extends T>` when you need to **read** (produce) from a collection
- Use `<? super T>` when you need to **write** (consume) to a collection
- Use `<?>` when you don't care about reading or writing

---

7. INHERITANCE & SUBTYPES

Inheritance with Generic Types

Even though `Integer` is a subclass of `Number`, `List<Integer>` is **NOT** a subclass of `List<Number>`. Generics are invariant, not covariant. This is called 'type invariance' in Java.

Why Type Invariance Exists

Consider this dangerous scenario if invariance was allowed:

```java
// If List<Integer> were a List<Number>:
List<Integer> intList = new ArrayList<>();
List<Number> numList = intList;  // Would be allowed if invariant
numList.add(3.14);  // Add a Double!
Integer num = intList.get(0); // ClassCastException!
// num is now 3.14, which is not an Integer!
```

This would break type safety! So Java doesn't allow this assignment.

Using Wildcards for Flexibility

To work around invariance, use wildcards:

```java
List<Integer> intList = new ArrayList<>();
List<? extends Number> numList = intList; // OK!
// numList is read-only (can't add elements)
for (Number n : numList) {
    System.out.println(n);
}
```

This is safe because we can only read elements as Numbers, preventing the problem above.

---

8. GENERIC SUPERCLASS AND SUBCLASS

Extending Generic Classes

A class can extend a generic superclass and either keep it generic, specify concrete types, or add its own type parameters.

Pattern 1: Subclass Keeps the Generic Type Parameter

```java
public class Animal<T> {
    protected T data;
    public void setData(T data) { this.data = data; }
    public T getData() { return data; }
}

public class Dog<T> extends Animal<T> {
    public void bark() {
        System.out.println("Dog barks");
    }
}

// Usage:
Dog<String> dog = new Dog<>();
dog.setData("Labrador");
```

Explanation: The subclass `Dog` also remains generic with type parameter T.

Pattern 2: Subclass Specifies a Concrete Type

```java
public class StringAnimal extends Animal<String> {
    // T is now String for this subclass
    public void printData() {
        System.out.println("Data: " + data.toUpperCase());
    }
}

// Usage:
StringAnimal strAnimal = new StringAnimal();
strAnimal.setData("Hello");
```

Explanation: The subclass fixes T to `String`. Now `setData()` only accepts String, and `getData()` always returns String.

Pattern 3: Subclass Adds Additional Type Parameters

```java
public class Pair<T, U> extends Animal<T> {
    private U secondValue;
    public void setSecond(U val) { this.secondValue = val; }
    public U getSecond() { return secondValue; }
}

// Usage:
Pair<String, Integer> pair = new Pair<>();
pair.setData("Name");
pair.setSecond(25);
```

Explanation: The subclass has both T and U type parameters. T is inherited from Animal, and U is new.

---

9. TYPE INFERENCE

What is Type Inference?

Type inference is the compiler's ability to determine the type parameters of a generic method or class based on the context where it is used. You don't always need to explicitly specify type parameters; the compiler can figure them out.

The Diamond Operator <>

Introduced in Java 7, the diamond operator <> allows the compiler to infer type arguments from the context.

Before Java 7 - Without Diamond Operator:

```java
// Java 5-6: Must explicitly specify types
List<String> list = new ArrayList<String>();
Box<Integer> box = new Box<Integer>();
```

Java 7+ - With Diamond Operator:

```java
// Java 7+: Compiler infers types
List<String> list = new ArrayList<>();
Box<Integer> box = new Box<>();

// The compiler sees that list is List<String>
// and infers that ArrayList must be ArrayList<String>
```

Type Inference in Generic Methods

The compiler infers method type parameters from the arguments passed.

```java
public static <T> T getFirst(T a, T b) {
    return a;
}

// Explicit type argument:
String result1 = Utils.<String>getFirst("A", "B");

// Compiler infers T is String from arguments:
String result2 = Utils.getFirst("A", "B");
```

Explanation: Both versions are valid. The compiler infers that T = String from the string arguments.

Type Inference with Method Chaining

```java
public static <T> Box<T> createBox(T item) {
    return new Box<T>();
}

// The compiler infers the return type from the context
List<Box<String>> list = new ArrayList<>();
list.add(createBox("Hello"));  // T inferred as String
list.add(createBox(42));  // This would be T inferred as Integer
```

---

10. RESTRICTIONS ON GENERICS

Why Are There Restrictions?

Generics are subject to restrictions to maintain backward compatibility with older Java versions (pre-Java 5) and due to type erasure. Java implements generics using compile-time checking and type erasure (removing type information at runtime).

Restriction 1: Cannot Create Instance of Type Parameter

You cannot instantiate a generic type directly.

WRONG:

```java
```

```
public class Box<T> {
    public void create() {
        T item = new T();  // COMPILE ERROR!
    }
}
```

Reason: At runtime, T is erased and the JVM doesn't know which class to instantiate.

CORRECT - Workaround using Class Parameter:

```
java

public class Box<T> {
    private Class<T> clazz;
    public Box(Class<T> clazz) { this.clazz = clazz; }
    public T create() throws Exception {
        return clazz.getDeclaredConstructor().newInstance();
    }
}

// Usage:
Box<String> box = new Box<>(String.class);
```

Explanation: By passing the Class object, we provide the runtime type information that was erased from the generic type.

Restriction 2: Cannot Create Array of Generic Type

You cannot create arrays of generic types directly.

WRONG:

```
java

List<String>[] array = new List<String>[10]; // ERROR!
```

Reason: Type erasure. Arrays store runtime type information, but generics are compile-time only. At runtime, the array would be `List[]`, and you could potentially add `List<Integer>` to it, breaking type safety.

CORRECT - Workaround using ArrayList:

```
java

List<List<String>> lists = new ArrayList<>();
```

Explanation: Use collections instead of arrays for generic types.

Restriction 3: Cannot Use Primitives as Type Arguments

Type parameters must be reference types (classes), not primitives.

WRONG:

```java
Box<int> box = new Box<>();      // ERROR!
List<double> list = new ArrayList<>(); // ERROR!
```

Reason: Type parameters must be object types. Primitives are not objects in Java.

CORRECT - Use Wrapper Classes:

```java
Box<Integer> box = new Box<>();
List<Double> list = new ArrayList<>();

// Java auto-boxing handles conversion
box.set(42);       // 42 is auto-boxed to Integer
int num = box.get(); // Result auto-unboxed to int
```

Restriction 4: Cannot Declare Static Fields of Type Parameter

Static fields cannot use the class's type parameters.

WRONG:

```java
public class Box<T> {
    private static T item;  // ERROR!
}
```

Reason: Static fields are shared across all instances. Since T is instance-specific, it cannot be static. What would the type be for all instances?

CORRECT:

```java
public class Box<T> {
    private T instanceItem;  // OK - instance field
    private static int count; // OK - no type parameter
}
```

Restriction 5: Cannot Use instanceof with Generic Types

You cannot use instanceof with parameterized types due to type erasure.

WRONG:

```java
if (obj instanceof List<String>) { }  // ERROR!
```

Reason: At runtime, `List<String>` becomes just `List` due to type erasure. The compiler cannot check the type parameter.

CORRECT - Use Raw Type or Wildcard:

```java
if (obj instanceof List) {  // Check raw type
    List<?> list = (List<?>) obj;
    // Now work with it
}
```

Restriction 6: Cannot Use Type Parameters in catch or throw

You cannot catch or throw exceptions using type parameters.

WRONG:

```java
public <T extends Exception> void doSomething() {
    try { } catch (T e) { }  // ERROR!
    throw new T();  // ERROR!
}
```

Reason: Exception handling requires knowing the exact exception type at runtime, which generics don't provide due to type erasure.

Restriction 7: Cannot Use Type Parameters in instanceof or cast

```java
public <T> void process(Object obj) {
    if (obj instanceof T) { }      // ERROR!
    T item = (T) obj;              // Warning - unchecked cast
}
```

Reason: At runtime, T is erased to Object, making this meaningless.

KEY EXAM TIPS & SUMMARY

Important Points to Remember:

1. **Generics are compile-time only**: Type information is erased at runtime through type erasure.
2. **Type Safety**: Use generics to catch errors at compile time, not runtime.
3. **Wildcards provide flexibility**: Use `<? extends T>` for reading, `<? super T>` for writing.
4. **Bounded types restrict possibilities**: Use `extends` to limit which types can be used.
5. **Diamond operator simplifies code**: Use `<>` instead of repeating type parameters (Java 7+).
6. **Type parameters are invariant**: `List<Integer>` is NOT a subclass of `List<Number>`.
7. **Methods can be generic**: Even non-generic classes can have generic methods.
8. **PECS Rule**: Producer Extends, Consumer Super.

Quick Reference Table:

Concept	Syntax	Purpose
Generic Class	`class Box<T> { }`	Create type-safe classes
Generic Method	`public <T> T method(T arg)`	Type-safe methods
Bounded Type	`<T extends Number>`	Restrict type range
Upper Wildcard	`<? extends Number>`	Accept subclasses, read-only
Lower Wildcard	`<? super Integer>`	Accept superclasses, write-only
Unbounded Wildcard	`<?>`	Accept any type
Diamond Operator	`new ArrayList<>()`	Type inference (Java 7+)

Most Common Exam Questions:

1. Explain the difference between type parameters and type arguments
2. Why can't you create arrays of generic types?
3. What is type erasure and how does it affect generics?
4. When should you use upper vs lower bounded wildcards?
5. How do PECS rule and type invariance work?
6. What are the limitations of generics and why do they exist?
7. Write code using bounded type parameters
8. Explain inheritance with generic classes

PRACTICE SCENARIOS

Scenario 1: Design a Generic Stack

java

```java
public class Stack<T> {
    private List<T> elements = new ArrayList<>();

    public void push(T element) {
        elements.add(element);
    }

    public T pop() {
        if (isEmpty()) throw new EmptyStackException();
        return elements.remove(elements.size() - 1);
    }

    public boolean isEmpty() {
        return elements.isEmpty();
    }
}
```

Scenario 2: Generic Method to Find Maximum

java

```java
public static <T extends Comparable<T>> T findMax(T[] array) {
    T max = array[0];
    for (T item : array) {
        if (item.compareTo(max) > 0) {
            max = item;
        }
    }
    return max;
}

// Usage:
Integer maxInt = findMax(new Integer[]{3, 1, 4, 1, 5});
String maxStr = findMax(new String[]{"apple", "zebra", "banana"});
```

Scenario 3: Working with Multiple Bounds

java

```
public class Comparable<T> {
    public static <T extends Comparable<T> & Cloneable>
    T duplicate(T original) throws Exception {
        // T must implement both Comparable and Cloneable
        return (T) ((Cloneable) original).clone();
    }
}
```

This guide covers all the fundamental concepts needed for your exam. Study each section carefully, understand the WHY behind each restriction, and practice writing code with each concept.